第 2 回目　Jetson による Airoco，カメラ，回路部品へのアクセス

2.1 概要

IoT デバイスでは，省電力，省スペースのために，一般的にネットワーク接続機能を有するマイコンが使われる．本実験では，マイコンの一例として Jetson Nano を用い，Airoco へのアクセスや，電子部品との接続を習得する．

2.2 Jetson とは

Jetson は NVIDIA 社の組み込みシングルボードコンピュータのシリーズである．機械学習アプリケーションを目的として設計され，画像認識，機械学習や自動運転などを高速に実行するための，GPU を備えた並列処理用の組込み用コンピューティングボードである．Jetson Nano は Jetson ファミリーの 1 機種である．

Jetson 向けのソフトウェアパッケージとして NVIDIA より JetPack が提供されている．これには，Linux ディストリビューションの Ubuntu をベースにしたデスクトップ環境のほか，Jetson の能力を用いた GPU コンピューティング，マルチメディア，グラフィックス，コンピュータービジョンのアクセラレーション用ライブラリー式が含まれている．

2.3 Jetson の基本設定

本実験で手渡す Jetson には SD カードに Jetson Nano Developer Kit SD Card Image ver. 4.6.1 がインストールしてある．さらに，OS の最低限の設定

- OS の表示言語として日本語選択（ただし，現在は日本語入力の IME は使えない）
- システムを 2025/4/2 の時点で最新版に更新
- NTP サービスの登録
- IP アドレスの割当
- VNC サーバーの登録

が済んでいる．ユーザー名やパスワードは「nano」を設定してある．ただし，VNC のパスワードは「nanonano」である（長さ制限への対応）．電源を入れて立ち上げれば，自動でログインまで進み，決められた IP アドレスが割り当てられる．また，Web ブラウザ（Chromium）で Google ドライブにアクセスすることで，他の PC とのファイル共有も可能である．なお，ユーザー名やパスワードを含めて，各種設定は使用者の必要に応じて変更して良い．実験期間終了後には，SD カードは新しいイメージで上書きされる．

各自の Jetson の IP アドレスは，別ファイル「Jetson_IP アドレス_先進プロジェクト実験 I 用.xlsx」を参照する．PC 側も同じ実験室ネットワークに接続後，VNC でアクセスできるようになる．なお，Jetson のボードには無線 LAN の機能は搭載されていないので，LAN ケーブルで接続する．

Jetson については「情報工学実験 I」の「コンピュータの仕組み」で基本を学習しているので，ここでは使用方法について簡単にヒントを挙げる．使いやすいように，各自の好みに合わせて設定する．

(a) 操作は基本的には VNC 接続で行う．PC 側の VNC クライアントとしては，Real VNC Viewer や UltraVNC Viewer などが使える．ただし，レスポンスの悪さが気になる場合は，HDMI ポートに外部ディスプレイを接続したり，USB ポートにキーボードやマウスを接続したりして使用することもできる．

また，VNC では今のところサーバーとクライアント間でクリップボードの共有ができないので，長いコマンドを入れたい場合はテキストファイルに書き込んで PC と共有する方法が使える．（(f)で紹介）

(b) システムのアップデートは，ディスクイメージ作成時点で行っているが，最新にしたければ以下のコマンドを入力する．（状況によっては，時間かかる可能性がある）

```
$ sudo apt update
$ sudo apt upgrade
$ sudo apt auto-remove
```

(c) JetPack には標準テキストエディタとして，gedit や vi がインストールされている．好みに応じて nano などのエディタをインストールする．

例） nano のインストール

```
$ sudo apt install nano
```

(d) メニューやフォルダ名を英語にしたい場合は，右上の歯車アイコンから「システム設定…」を選び，言語サポートを選択，システムから「言語サポートが完全にはインストールされていません」と通知が出るが無視して良い．「メニューとウィンドウの言語：」で「English」を選択して「Apply System-Wide」ボタンを押す．また，「キーボード入力に使う IM システム」は「iBus」を選択する．，

(e) 英語システムから日本語システムにする場合の設定（最初の状態は適用済み）

以前はできていた IME を使った日本語入力が（アップデートが理由かも）できなくなったので，基本的に英語で使用する．どうしても日本語を入力したい場合は仮想キーボードが使える．また，日本語フォルダ名を入力したい場合は，Copy&Paste が使える．今後，日本語入力が可能な方法が見つかったらお知らせする．（2024.1.31）

右上の歯車アイコンから「System settings…」を選び，「Language Support」を選択，アップデートが必要と出たら，指示に従ってインストールする．その後，言語一覧から「日本語」を選択し一番上にドラッグしてから，「システム全体に適用」をクリックする．「言語のインストールと削除…」で日本語をインストールし，必要に応じて再起動する．「キーボード入力に使う IM システム」を「iBus」から「fcitx」へ変更する．再起動後に，Ctrl+[Space]キーか「半角/全角」キーで日本語入力モードとなる．入力モードの切り替えは，右上のキーボードアイコン(Fig. 1)でも可能である．

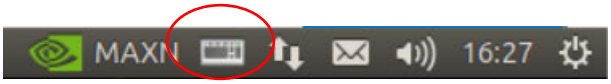


Fig. 1　画面右上のキーボードアイコン

(f) PC とのファイル共有方法

ウェブブラウザ Chromium に大学のメールアドレスでログインして Google ドライブを開けば，PC とのファイル交換が可能である．

さらに便利に使いたい場合，Samba をインストールして，これでファイル共有をする方法もある．

Samba 設定方法：

まず，以下のコマンドでインストールする．

```
$ sudo apt install samba
```

日本語 OS にしてフォルダ名が「公開」になっている場合，必要に応じて使い易いように英語名に戻す．

```
$ mv ~/公開 ~/Public
```

設定ファイル /etc/samba/smb.conf に以下の内容を追加（管理者権限で変更する必要がある）．この後の説明の「ユーザー名」は手渡された状態のまま使っている場合は「nano」に変える．

```
[jetson]
path = /home/ユーザー名/Public
read only = no
```

```
guest ok = yes
public = yes
writable = Yes
browsable = yes
force user = ユーザー名
directory mode = 0774
create mode = 0774
```

その後以下を実行.

```
$ sudo pdbedit -a ユーザー名   # Samba のユーザー登録を行う
$ sudo service smbd restart
$ sudo service nmbd restart
```

(g) 画面のブランクアウトの時間は，右上歯車アイコンから「システム設定…」を選び，「画面の明るさとロック」から設定できる.

(h) キーボードの CapsLock キーと Ctrl キーを入れ替えた方が使いやすい場合は以下のコマンドで設定できる.

```
$ gsettings set org.gnome.desktop.input-sources xkb-options "['ctrl:swapcaps']"
```

なお，以下のコマンドでリセットできる.

```
$ gsettings reset org.gnome.desktop.input-sources xkb-options
```

2.4 Airoco へのアクセス

まずは，第 1 回の JavaScript プログラム(sample1, 2)を Jetson にコピーして，Web ブラウザの Chromium で動作確認する.

次に，Python プログラム（sample3, 4）を「ipynb」形式から純粋な Python で動かせるテキストファイルの「py」形式に変換し，Jetson にコピーして Python3 で動作確認する.

例 `$ python3 sample1-3.py`

なお，sample1-4.py のグラフのウィンドウは，1つ閉じると次が表示される.

2.5 USB カメラを用いたリアルタイム物体検出

物体検出をニューラルネットワークで行うアルゴリズム YOLO(You Only Look Once)を用いたプログラム「Tiny YOLO v2」をインストールして，USB カメラを用いたリアルタイム物体検出を試す．このプログラムでは，以下の 20 種類の認識が可能である．実際に物体検出している様子を Fig. 2 に示す.

- 人
- 動物（鳥，猫，牛，犬，馬，羊）
- 乗り物（航空機，自転車，ボート，バス，自動車，オートバイ，列車）
- 屋内（瓶，椅子，食卓，鉢植えの植物，ソファー，テレビモニター）

以下の手順でインストールする．最後のコマンドはおそらく 20 分以上かかるので，その間に次の回路操作のブレッドボードの準備などを行う.

```
$ sudo apt update
$ sudo apt install python3-pip protobuf-compiler libprotoc-dev libjpeg-dev cmake
```

```
$ pip3 install --user cython
$ git clone https://github.com/tsutof/tiny_yolov2_onnx_cam
$ cd tiny_yolov2_onnx_cam
$ export PATH=$PATH:/usr/local/cuda/bin
$ python3 -m pip install -r requirements.txt   # 状況によるが 20 分程度かかる
```

本来は不要なはずだが，ダウンロード先が壊れているようなので，提供された
tinyyolov2-8.tar.gz
を ~/tiny_yolov2_onnx_cam にコピーする．

実行のために最大クロックにする．

```
$ sudo nvpmodel -m 0
$ sudo jetson_clocks
```

USB カメラを接続して，~/tiny_yolov2_onnx_cam ディレクトリで以下のコマンドで実行．初回はダウンロードとビルドを行うので時間がかかる．

```
$ python3 tiny_yolov2_onnx_cam.py --camera 0
```

なお，

```
$ python3 tiny_yolov2_onnx_cam.py -h
```

ですべてのオプションが表示される．

Fig. 2 リアルタイム物体検出の様子

参考： Jetson Japan User Group 著，Jetson Nano 超入門 改定第2版，ソーテック社，2021

2.6 Jetson の GPIO による回路操作

GPIO(General-Purpose Input/Output)は「汎用入出力」という意味であり，プログラムによって入出力が可能な端子である．Jeson Nano には 40 ピンの拡張ヘッダーがあり，電源ピン（+3.3V，+5.0V，GND），特定の通信機能のピン（I2C，

UART)と，それ以外の GPIO に使えるピンから構成されている． ピン番号と機能を Fig. 3 に示す．例えば，3.3V を取り出したいときには 3.3VDC に，グラウンドにつなぎたいときには GND に接続する．GPIO と記載のピンのオン・オフをプログラムから制御できる．

SoC GPIO	Linux GPIO #	Alternate Function	Default Function			Default Function	Alternate Function	Linux GPIO #	SoC GPIO
			3.3 VDC	1	2	5 VDC			
PJ.03	75	GPIO	I2C1_SDA	3	4	5 VDC			
PJ.02	74	GPIO	I2C1_SCL	5	6	GND			
PBB.00	216	AUD_CLK	GPIO	7	8	UART1_TXD	GPIO	48	PG.00
			GND	9	10	UART1_RXD	GPIO	49	PG.01
PG.02	50	UART1_RTS	GPIO	11	12	GPIO	I2S0_SCLK	79	PJ.07
PB.06	14	SPI1_SCK	GPIO	13	14	GND			
PY.02	194		GPIO	15	16	GPIO	SPI1_CS1	232	PDD.00
			3.3 VDC	17	18	GPIO	SPI1_CS0	15	PB.07
PC.00	16	SPI0_MOSI	GPIO	19	20	GND			
PC.01	17	SPI0_MISO	GPIO	21	22	GPIO	SPI1_MISO	13	PB.05
PC.02	18	SPI0_SCK	GPIO	23	24	GPIO	SPI0_CS0	19	PC.03
			GND	25	26	GPIO	SPI0_CS1	20	PC.04
PB.05	13	GPIO	I2C0_SDA	27	28	I2C0_CLK	GPIO	18	PC.02
PS.05	149	CAM_MCLK	GPIO	29	30	GND			
PZ.00	200	CAM_MCLK	GPIO	31	32	GPIO	PWM	168	PV.00
PE.06	38	PWM	GPIO	33	34	GND			
PJ.04	76	I2S0_FS	GPIO	35	36	GPIO	UART1_CTS	51	PG.03
PB.04	12	SPI1_MOSI	GPIO	37	38	GPIO	I2S0_DIN	77	PJ.05
			GND	39	40	GPIO	I2S0_DOUT	78	PJ.06

Fig. 3　Jetson Nano の拡張コネクタ

NVIDIA が Jetson.GPIO ライブラリを提供している． Jetson.GPIO は「Raspberry Pi」の GPIO パッケージ（RPi.GPIO）とインタフェースが互換になっているので，Python でプログラムする場合，同じ書き方ができる．

Jetson Nano で GPIO を使用するためには Jetson GPIO ライブラリをインストールする必要がある．ただし，新しいバージョン（例えば，v.2.1.0. 2023 年 4 月現在）だと不具合があるため，以前のバージョン（v2.0.21）をインストールする．

```
$ pip3 install Jetson.GPIO==2.0.21
```

Jetson GPIO ライブラリを使用するために，以下の設定を行う．ユーザー名部分は，変更していなければ nano とする．

```
$ sudo groupadd -f -r gpio
$ sudo usermod -a -G gpio ユーザー名
$ sudo cp ~/.local/lib/python3.6/site-packages/Jetson/GPIO/99-gpio.rules
    /etc/udev/rules.d/
     # 上は 2 行に見えるが，1 行でスペース部分で見かけ上改行されている
$ sudo udevadm control --reload-rules && sudo udevadm trigger
$ sudo reboot      # 再起動されることに注意．
```

次に，以下のコマンドで，個人のディレクトリにサンプルプログラムをコピーする．

```
$ sudo cp -r /usr/share/doc/jetson-gpio-common ~
$ cd ~
$ mv jetson-gpio-common jetson-gpio
$ cd jetson-gpio/samples
```

この後，GPIO のサンプルプログラムを実行するときには，このディレクトリのものを使用する．

Jetson の GPIO は Raspberry Pi の GPIO と互換になっていて，プログラムもほぼ変更なしに使うことができる．各ピンを指定する方法が 2 つ用意されていて，サンプルプログラム（例えば，simple_out.py を参照）の

```
GPIO.setmode(GPIO.BCM)
```

で，BCM ピン番号を使う指定方法になる．Fig. 3 のピン番号にしたいときには，

```
GPIO.setmode(GPIO.BOARD)
```

とすれば良い(物理ピン番号と呼ばれる)．BCM ピン番号の 18 番が物理ピン番号の 12 番に対応している．

Jetson Nano の GPIO ピンは，オンの電圧 3.3V で，流せる電流の制限は非常に小さい（一説には 1 ピン 1mA 程度，はっきりした記述が見つからない）． LED は駆動できるが，ブザーを鳴らしたり，モータを動かしたりするのは電流制限のためできない．参考までに，Raspberry Pi の電流制限は 1 ピンあたり 16mA である．

- LED の点灯制御（L チカ）

Fig. 4 のような回路を作成する．LED のカソード側を GND（6 ピン）に，アノード側を GPIO（12 ピン）に接続して，GPIO をオン・オフして点滅させる．抵抗器無しで LED を単独で接続すると電流制限の限界まで電流が流れて LED が破損するので注意する（Jetson の場合は電流制限で大丈夫かもしれないが，一般的に抵抗を入れる）．掛ける電圧と流す電流から LED に直列に入れる抵抗値を決める．今回は，回路図の通りにすればよい．LED の順方向電圧降下を 0.6V とすると

$$\frac{3.3-0.6}{220} \approx 0.013$$

なので，13mA 程度流れることになる（ハードウェアで流せる限界を超える場合はそちらが優先される）．この値が LED の電流の最大定格を超えないように注意する．

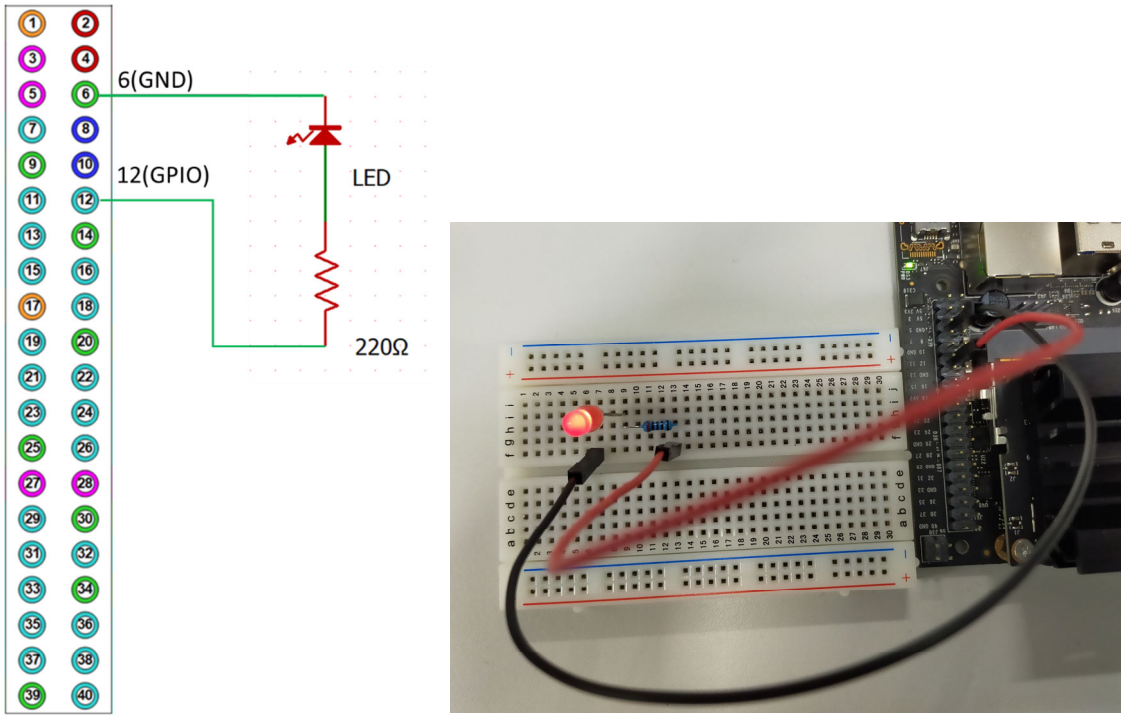


Fig. 4　LED 点灯制御の回路

回路作成には，ブレッドボードを用いる．ブレッドボードは，差し込むことでワイヤーや部品を接続できる．ブレッドボードのピンは Fig. 5 に示す部分で内部接続している．なお，一般的に，赤線は電源用，青線は GND 用に用いられる．

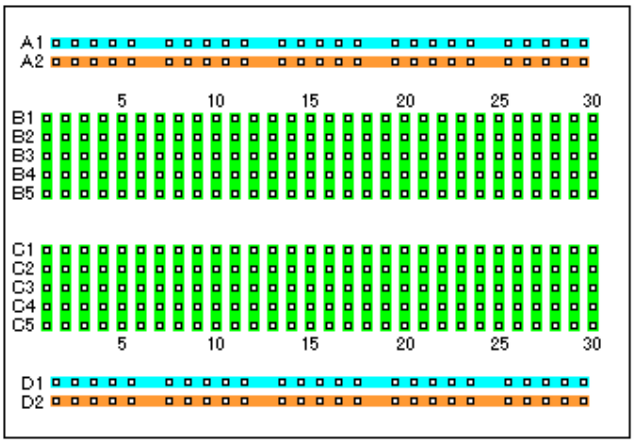


Fig. 5　ブレッドボードの内部接続

GPIO のサンプルプログラムディレクトリの simple_out.py をそのままで実行し，LED を点滅させる．

```
$ python3 simple_out.py
```

サンプルプログラムでは，1 秒間隔で GPIO の 12 ピンの出力をオン，オフしている．

- 電子ブザーを鳴らす

Fig. 6 のように回路を組み替えたあと，再度 GPIO サンプルのディレクトリの simple_out.py を実行し，1 秒間隔でブザー（PB04-SE12HPR）をオン，オフする．GPIO から流せる電流ではブザーを駆動できないので，トランジスタを入れてブザーへの電流を制御する回路とする．図に従って回路を組めば良い．S8050 は NPN バイポーラトランジスタである．端子の機能の違い（Fig. 7）に注意して，向きに注意しながら回路を組む．電子ブザーにも極性（長い方が＋）がある．間違えると破損する可能性があるので注意する．

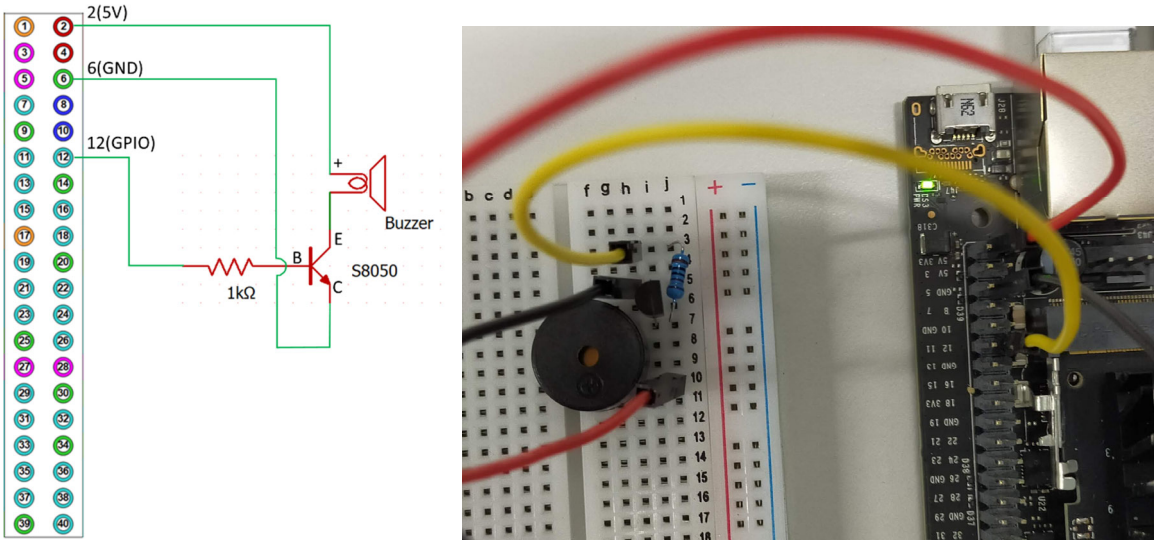


Fig. 6　電子ブザー用回路

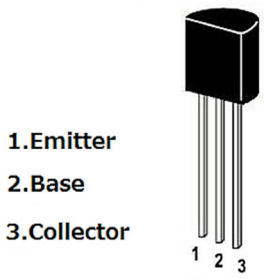


Fig. 7　NPN トランジスタ S8050

- I2C 接続のセンサから情報を取得

I2C は IC 間の通信方法の一つである．電源，GND のほかにクロック(SCL)とデータ(SDA)の 2 線，合計4本のケーブルで通信が可能である．I2C 機器はアドレスを持っていて区別できるので，複数の I2C 機器を数珠繋ぎに接続することができる．

ここでは，温度，湿度，気圧を取得できる BOSCH BME280 使用のセンサモジュールキットを用いる．詳細はモジュールキットの取り扱い説明書を見る（あらかじめ，基板にピンヘッダと J1,J2,J3 がはんだ付けしてある）．必要に応じて，BME280 のデータシートも参照する．

Fig. 8 のように配線する．

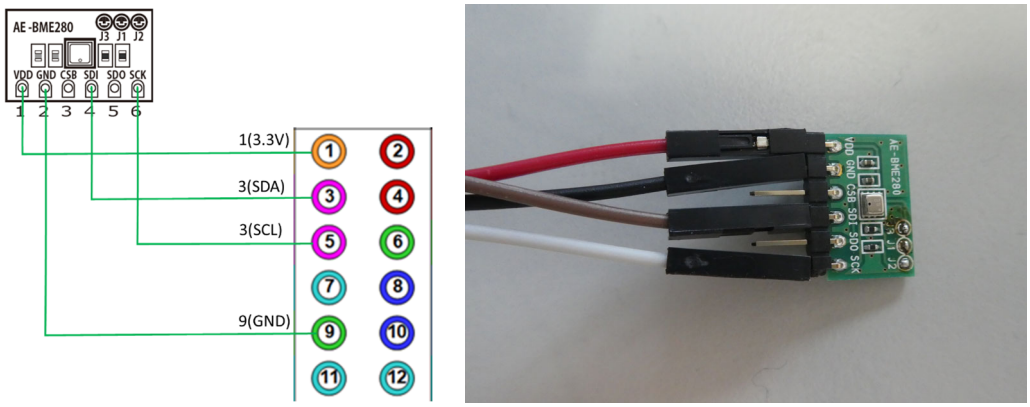
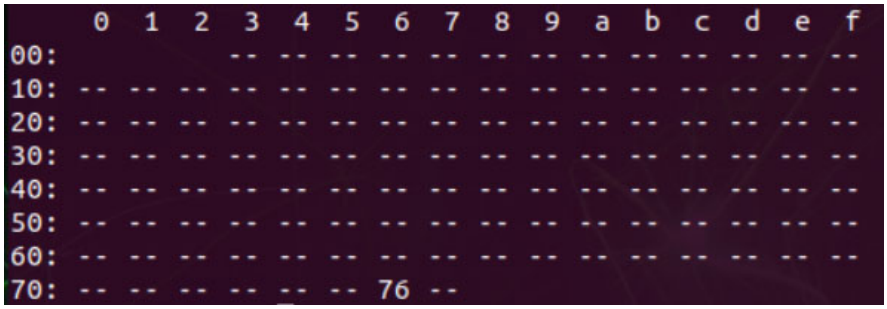


Fig. 8　BME280 センサモジュール用の配線

以下のコマンドで，I2C ピンに接続しているチップを確認できる．

```
$ sudo i2cdetect -r -y 1
```

BME280 は 0x76 をアドレスとして持つので，Fig. 9 のような出力になれば良い．



```
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- 76 --
```

Fig. 9　BME280 の接続確認

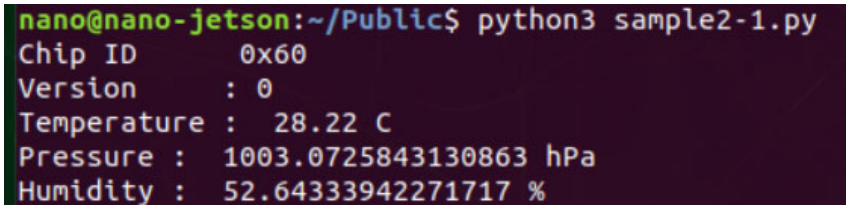
サンプルプログラム sample2-1.py を動かすために，プログラムで用いるライブラリのインストールを行う．

```
$ pip3 install smbus2
```

次に，提供されたプログラムを使って

```
$ python3 sample2-1.py
```

で出力を確認する．うまく行っていれば，Fig. 10 のような出力が得られる．



```
nano@nano-jetson:~/Public$ python3 sample2-1.py
Chip ID       0x60
Version      : 0
Temperature :  28.22 C
Pressure :  1003.0725843130863 hPa
Humidity :  52.64333942271717 %
```

Fig. 10　BME280 からの出力例

参考:

Jetson.GPIO ライブラリ公式ページ

https://github.com/NVIDIA/jetson-gpio

Jetson ハードウェア情報

https://developer.nvidia.com/embedded/learn/jetson-nano-2gb-devkit-user-guide?ref=morisinoblog

2.7 課題

【基本課題 2-1】2.6 節の「LED の点灯制御」で，サンプルプログラム simple_out.py の

```
GPIO.setmode(GPIO.BCM)
```

を

```
GPIO.setmode(GPIO.BOARD)
```

に書き換えた上で LED の点灯を実現せよ．（ファイルを書き換える際には，権限が必要なことに注意）

【基本課題 2-2】 Airoco のセンサの CO2 濃度が予め設定したしきい値を超えたらブザーを鳴らすシステムを構築せよ．

【発展課題 2-1】 LED 点滅を 0.5 秒周期，ブザーオン・オフを 1 秒周期で同時に行う回路とプログラムを作成せよ.